

SUPPLEMENTARY · MODULE 2

Microservices Design and REST APIs

Design REST APIs that teams can evolve: resource models, status codes, idempotency, OpenAPI as the contract, and the patterns that keep consumers from coupling to your internals.



FROM BUSINESS REQUIREMENTS TO SERVICE DESIGN · DOMAIN-DRIVEN DESIGN AS A SERVICE DESIGN TOOL



THE SEVEN-STEP PATH

1. Understand the business requirement
2. Identify business capabilities
3. Define the domain and bounded contexts
4. Identify service boundaries
5. Design service responsibilities and contracts
6. Choose a communication and data strategy
7. Validate and refine the design



WHY START WITH BUSINESS REQUIREMENTS

- Technology-first design produces services that mirror database tables, not business capabilities
- **Requirements reveal the true nouns of the domain:** Customer, Account, Transaction, Loan
- Design validated against real workflows is far less likely to need costly rework later



CORE DDD CONCEPTS

- **Domain:** the overall business area a system supports
- **Subdomain:** a focused slice of the domain (core, supporting, or generic)
- **Bounded context:** a boundary where a model has one specific meaning
- **Ubiquitous language:** shared business terms used consistently inside a context
- **Aggregate:** a consistency boundary for related domain objects
- **Domain event:** a meaningful business fact, such as AccountOpened

KEY TAKEAWAY A seven-step path takes a business requirement from raw need to a validated, ready-to-build service design. • DDD concepts -- domain, bounded context, ubiquitous language, aggregate -- turn business understanding into service boundaries.

DOMAINS, SUBDOMAINS, AND BUSINESS CAPABILITIES ·

UNDERSTANDING BOUNDED CONTEXTS



DOMAIN HIERARCHY

- Domain -> Subdomain -> Business Capability -> Candidate Service
- **Banking domain example:** customer management, accounts, transactions, lending, fraud, notifications



WHAT A BOUNDED CONTEXT IS

- A boundary within which a domain model has one specific, consistent meaning
- Prevents teams from forcing a single shared model across the whole enterprise
- Bounded contexts usually align 1:1 with service boundaries



THREE KINDS OF SUBDOMAIN

- **Core:** the competitive differentiator -- invest heavily, build in-house
- **Supporting:** necessary but not differentiating -- build with moderate investment
- **Generic:** common, well-solved problems -- prefer buying or reusing



BANKING EXAMPLE: 'CUSTOMER' MEANS DIFFERENT THINGS

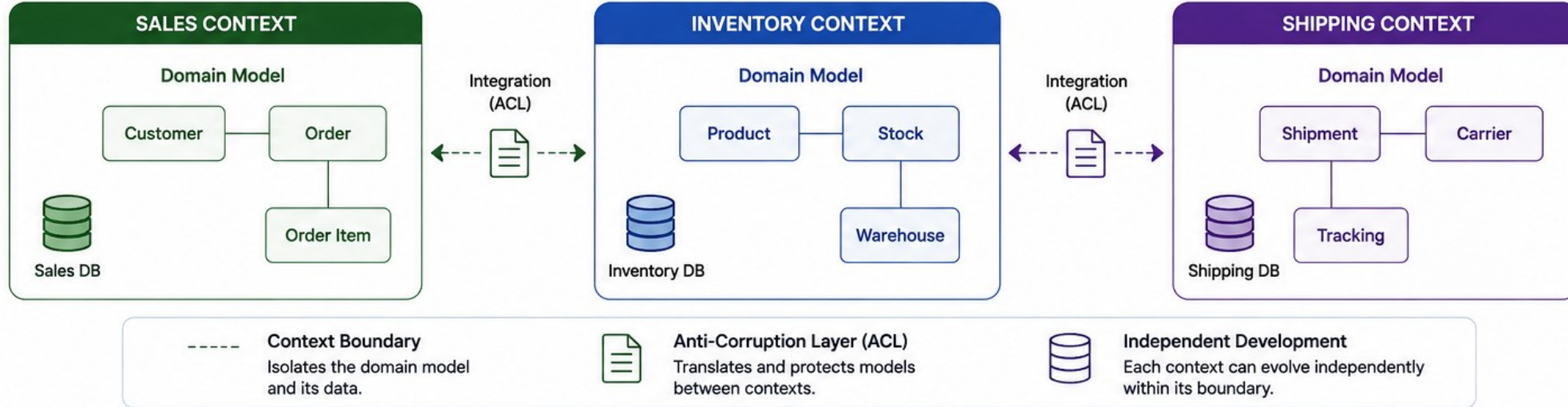
- **Account Service:** accounts, status machine, customerId as a field (no separate Customer Service)
- **Account Service:** a customer ID linked to accounts
- **Fraud Service:** risk signals and behavior patterns
- **Marketing Service:** preferences and campaign eligibility

KEY TAKEAWAY Core, supporting, and generic subdomains each deserve a different level of investment and a different build-vs-buy decision. • The word 'customer' means something different to Customer, Account, Fraud, and Marketing services -- each owns its own model.

BOUNDED CONTEXT AND SERVICE BOUNDARIES

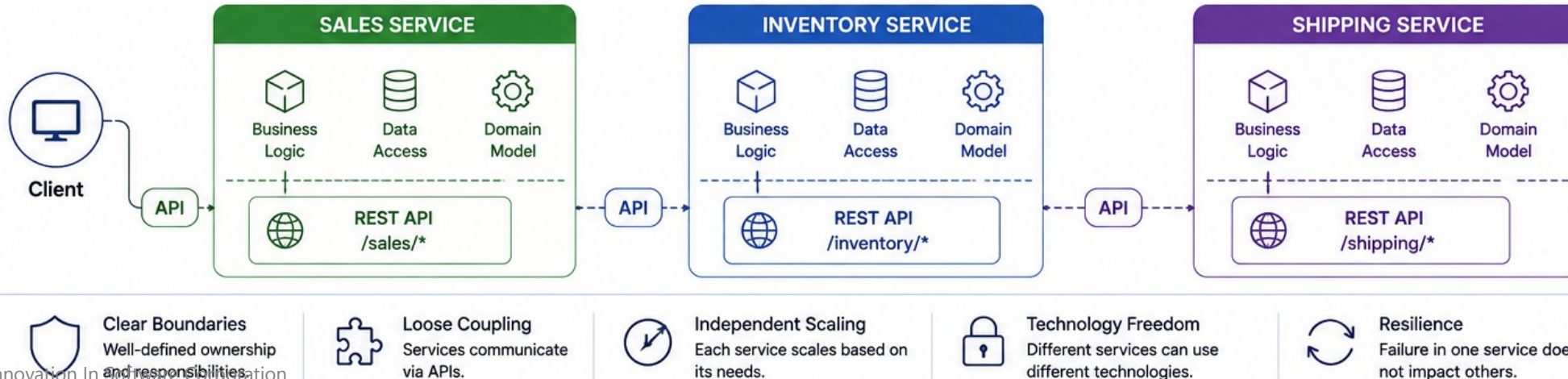
1. BOUNDED CONTEXT

A bounded context defines a clear boundary within which a specific domain model is defined and is not visible outside that boundary.



2. SERVICE BOUNDARIES

Each service owns a bounded context and exposes capabilities through well-defined APIs.



Context Mapping Between Services

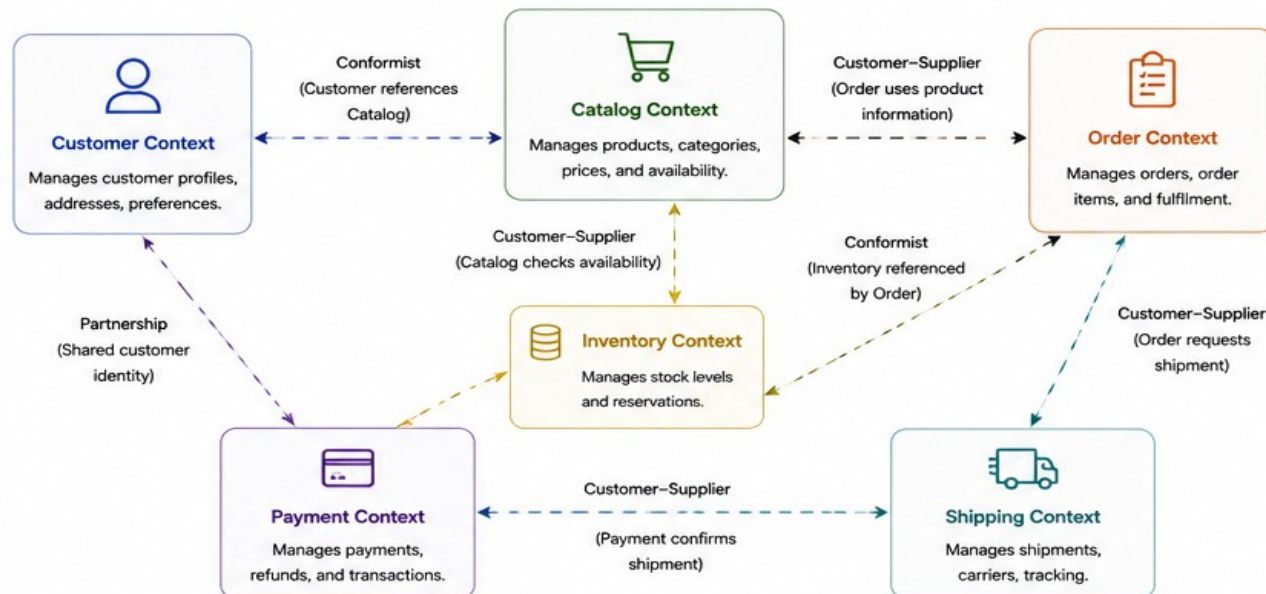
Context mapping helps teams visualize and manage relationships and translations between bounded contexts.



Why Context Mapping?

- Clarifies Relationships**
Understand how contexts relate and interact.
- Manages Translations**
Make differences in models explicit.
- Reduces Coupling**
Choose the right integration style for each relationship.
- Improves Collaboration**
Align teams around shared understanding.
- Supports Evolution**
Evolve contexts independently without breaking others.

Example: E-Commerce Platform Context Map



Legend – Relationship Types

- Partnership**
Two contexts collaborate closely with aligned goals.
- Customer-Supplier**
One context depends on another for information or services.
- Conformist**
One context conforms to another's model.
- Anti-Corruption Layer**
A translation layer protects a context from external model changes.

Common Relationship Types

- Partnership**: High collaboration, shared understanding, joint evolution.
- Customer-Supplier**: Upstream context provides a service or information.
- Conformist**: Downstream context adapts to upstream model.
- Anti-Corruption Layer**: A layer isolates context from external model changes.
- Shared Kernel (Avoid)**: Sharing models creates tight coupling and limits autonomy.

Example Translations

- Order Context** → **Catalog Context**: Translates `OrderItem.productId` → `Product.id`
- Payment Context** → **Order Context**: Translates `Order.total` → `Payment.amount`
- Shipping Context** → **Order Context**: Translates `Order.address` → `Shipping.address`
- Inventory Context** → **Order Context**: Translates `Availability` → `InventoryReservation`
- Customer Context** → **Payment Context**: Translates `Customer.id` → `Payment.customerRef`

Tips for Effective Context Mapping

- Start with the domain and identify key bounded contexts.
- Use a ubiquitous language and validate with domain experts.
- Keep contexts as independent as possible.
- Choose the simplest integration style that works.
- Document assumptions and translation rules.
- Review and evolve the map as the domain evolves.
- Goal**: Clear boundaries, explicit relationships, and independent evolution.

IDENTIFYING SERVICE BOUNDARIES · SERVICE GRANULARITY AND THE “RIGHT-SIZED” SERVICE



BOUNDARY-DESIGN QUESTIONS

- What business capability does this service own?
- What data does it own?
- Who owns the service (which team)?
- What APIs does it expose, and what events does it publish or consume?
- How often does it change, and does it need independent scaling?



THE GRANULARITY SPECTRUM

- **Too fine-grained:** excessive network calls, chatty integration, high operational overhead
- **Right-sized:** aligned to a business capability, independently deployable, manageable to own
- **Too coarse-grained:** a distributed monolith -- separate processes, still tightly coupled



WARNING SIGNS OF POOR BOUNDARIES

- Frequent cross-service changes required for a single feature
- Shared database tables between services
- Services that cannot be deployed independently
- Duplicated business rules across services



SIGNALS FOR RIGHT-SIZING

- Team can own, understand, and change the service independently
- Service has a single, cohesive business responsibility
- Data and API boundaries are clean, without leaking internals

KEY TAKEAWAY Six boundary-design questions -- capability, data, ownership, APIs, events, change rate -- turn a vague service idea into a defined one. • Right-sized services balance independent deployability against the operational overhead of running too many tiny services.

DECOMPOSING BY BUSINESS CAPABILITY · DECOMPOSING BY SUBDOMAIN

· DECOMPOSING AN EXISTING MONOLITH



CAPABILITY-BASED DECOMPOSITION

- List the business capabilities the organization performs, independent of org chart or technology
- **Each capability becomes a candidate service:** Customer Management, Account Management, Payments, Lending
- Capabilities are more stable over time than teams, technologies, or database schemas



SUBDOMAIN-BASED DECOMPOSITION

- Identify core subdomains (competitive advantage) and invest most heavily there
- Identify supporting subdomains and build them with moderate investment
- Identify generic subdomains and prefer buying, reusing, or outsourcing
- Each subdomain typically becomes one or more services with a clear owner



THE FOUR-STEP DECOMPOSITION PROCESS

1. Analyze the existing monolith's modules, data, and dependencies
2. Identify natural seams along business capability and data-ownership lines
3. Extract one service incrementally, starting with a low-risk, high-value seam
4. Validate the extraction against real workflows before repeating the process

KEY TAKEAWAY Mapping what the business does -- open accounts, process payments, manage loans -- produces natural, durable service boundaries. • Subdomain-based decomposition complements capability-based decomposition, weighting investment by business differentiation. • Analyze the monolith, identify seams, extract incrementally, and validate at each step -- decomposition is a journey, not an event.

THE STRANGLER FIG MIGRATION PATTERN · LOOSE COUPLING IN PRACTICE



FOUR-STAGE STRANGLER FIG MIGRATION

1. Place a routing layer (facade or gateway) in front of the monolith
2. Build new services for individual capabilities alongside the monolith
3. Redirect traffic for each capability to the new service as it becomes ready
4. Retire the corresponding monolith code once the new service fully replaces it



TIGHTLY COUPLED VS. LOOSELY COUPLED

- **Tightly coupled:** Account Service reads directly from `customer_db`
- **Loosely coupled:** Transaction Service calls the Account Service API or consumes account events



WHY IT WORKS

- The monolith keeps serving traffic throughout the migration -- low risk, reversible steps
- Each capability can be migrated, tested, and validated independently



COMMON COUPLING RISKS TO AVOID

- Database coupling -- sharing tables across services
- Deployment coupling -- services that must deploy together
- API contract coupling -- breaking changes ripple across consumers
- Temporal coupling -- one service must be up for another to finish work

KEY TAKEAWAY Route traffic incrementally to new services while the monolith keeps running, avoiding the risk of an all-at-once cutover. • Transaction Service should call Account Service's API, not reach into `account_db` -- coupling risk hides in shared data, not just shared code.

HIGH COHESION IN PRACTICE · AVOIDING SHARED BUSINESS LOGIC



LOW COHESION VS. HIGH COHESION

- **Low cohesion:** a BankingUtilityService mixing customer lookup, account update, email, reporting, and fraud scoring
- **High cohesion:** separate Customer, Account, Notification, Reporting, and Fraud Detection services



THE HIDDEN RISK OF SHARED LOGIC

- A shared library with domain rules becomes a de facto deployment dependency
- Changing the rule means coordinating releases across every consuming service
- Business rules belong inside the service that owns the corresponding capability



WHY IT MATTERS

- Cohesive services are easier to understand, test, own, and evolve
- Business-capability alignment is the most reliable path to high cohesion



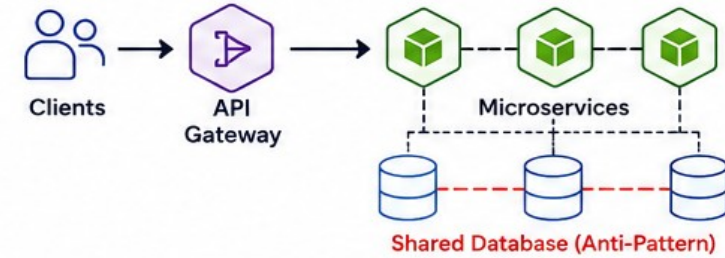
WHAT CAN SAFELY BE SHARED

- **Generic technical utilities:** logging, tracing, HTTP client wrappers
- **Not shared:** validation rules, pricing logic, eligibility rules, workflow decisions

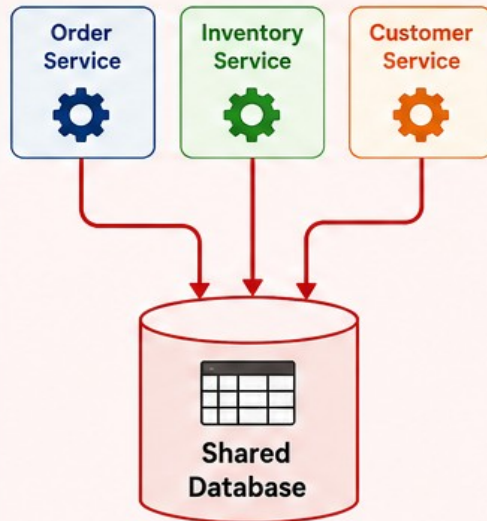
KEY TAKEAWAY Account Service should own account behavior end to end; a catch-all utility service confuses ownership and slows every change. • A shared library of business rules forces every consuming service to redeploy together -- the opposite of independent deployability.

Avoiding Shared Databases

Each microservice must own its data. Sharing a database couples services, creates hidden dependencies, and makes scaling, deployment, and evolution painful.

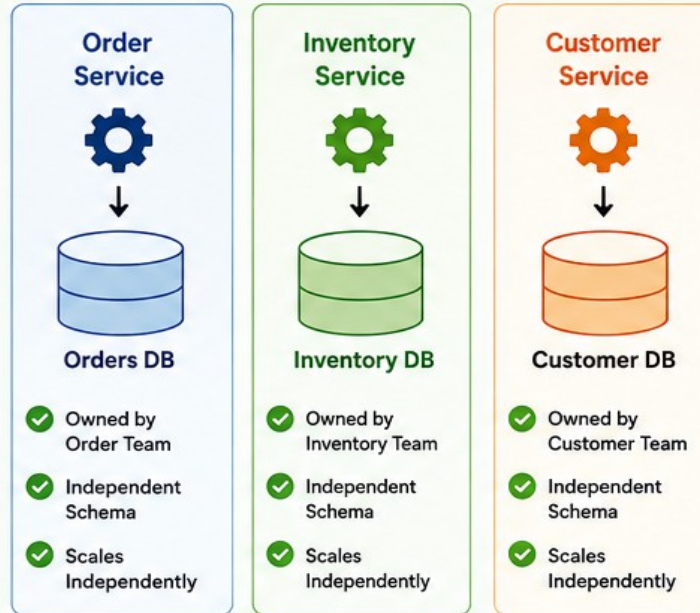


❌ THE ANTI-PATTERN: SHARED DATABASE



- TIGHT COUPLING**
Services become dependent on the same schema and data.
- DEPENDENCY HELL**
Schema changes ripple across services and teams.
- DEPLOYMENT BLOCKERS**
Teams cannot deploy independently.
- SCALING LIMITATIONS**
All services must scale together.
- RISK AND DATA INTEGRITY**
One service can accidentally impact another service's data.
- SLOWS DOWN EVOLUTION**
Hard to adopt new technologies or data models.

✅ THE PATTERN: DATABASE PER SERVICE



- LOOSE COUPLING**
Services are independent and autonomous.
- INDEPENDENT DEPLOYMENT**
Teams deploy and release without blocking others.
- FLEXIBLE SCALING**
Scale only what is needed.
- TECHNOLOGY FREEDOM**
Choose the best data store for each service.
- IMPROVED RESILIENCE**
Faults are isolated to a single service.



KEY TAKEAWAYS



Data is a product—
treat it as a first-class
service responsibility.



Ownership drives
accountability and
better decisions.



Independence enables
agility, reliability, and
continuous delivery.



Design for autonomy
today to avoid pain
tomorrow.

SERVICE OWNERSHIP AND GOVERNANCE · STATELESS VS. STATEFUL SERVICES



OWNERSHIP RESPONSIBILITIES

- A single team owns each service's code, data, deployment, and on-call responsibility
- Ownership includes documenting the API contract and its SLAs
- Ownership includes deciding when and how the service evolves



STATELESS SERVICES

- Do not rely on local, in-memory state from a previous request
- Easier to scale horizontally, restart, and load balance
- **State needed for correctness is externalized:** database, cache, token



LIGHTWEIGHT GOVERNANCE

- **Enterprise-wide standards:** naming, security, versioning, observability
- Governance should enable teams, not create a bottleneck for every decision



STATEFUL COMPONENTS

- Most business services are stateful in that they manage persistent data
- The question is where state lives -- not whether it exists
- Stateful components (PostgreSQL, Kafka) need backup and recovery plans; Redis is lecture-only in this course

KEY TAKEAWAY Every service needs a named owning team, a documented contract, and a small set of enterprise-wide standards it must follow.

- Stateless services keep no required local memory between requests; stateful components persist data and need extra operational care.

DESIGNING STATELESS MICROSERVICES · MANAGING STATE IN DISTRIBUTED APPLICATIONS



DESIGNING FOR STATELESSNESS

- Client sends request with token -> service validates token -> service reads required data from database/cache -> service returns response
- Use tokens for authentication context, not server-side session memory
- Store cache data in an external, shared cache when state must be shared



GOOD VS. RISKY STATEFUL DESIGN

- **Good:** Account Service pod -> Account database (survives pod restart)
- **Risky:** Account Service pod stores pending transactions only in memory (lost on failure)



COMMON MISTAKES

- Keeping user sessions only in memory
- Storing workflow state in local variables across requests
- Assuming requests always reach the same service instance



OPERATIONAL REQUIREMENTS FOR STATE

- Backup and recovery plans for every stateful dependency
- Monitoring for PostgreSQL, Kafka, and the three Spring Boot services
- Treat caches carefully -- cached state can become stale

KEY TAKEAWAY Any pod can serve any request when authentication uses tokens and persistent data lives in an external store, not local memory.

- A pod storing pending transactions only in memory loses that data the moment it fails -- durable state needs a durable home.

SYNCHRONOUS VS. ASYNCHRONOUS COMMUNICATION · CHOOSING A SERVICE COMMUNICATION STRATEGY



SYNCHRONOUS REQUEST-RESPONSE

- One service calls another and waits for a response
- Simple, testable, gives an immediate answer
- **Creates runtime dependency:** latency, timeouts, cascading failures



ASYNCHRONOUS MESSAGING

- A service publishes an event and does not wait for consumers
- Supports loose coupling, multiple consumers, and background processing
- Introduces eventual consistency and duplicate-message handling



DECISION FACTORS

- Does the caller need an immediate answer, or can it react later?
- How many consumers need to know about this event?
- Can the workflow tolerate eventual consistency?
- How critical is decoupling the producer from consumer availability?

KEY TAKEAWAY Choosing sync or async is a tradeoff between immediacy and coupling -- the right choice depends on what the business need actually is. • Immediate, user-facing answers favor REST; background reactions and fan-out favor messaging -- most systems need both.

API-BASED INTEGRATION · MESSAGING-BASED INTEGRATION



HOW API COMMUNICATION WORKS

- Transaction Service calls GET /accounts/5001 and waits for the response
- Widely supported, easy to test, compatible with REST and HTTP status codes



HOW MESSAGING COMMUNICATION WORKS

- Producer publishes a message or event through a broker
- Producer does not need to know when or whether each consumer processes it
- **Events are named in past tense:** CustomerCreated, PaymentFailed



REQUIRED SAFEGUARDS

- Timeouts, retries on safe operations only, and circuit breakers
- Correlation IDs, authentication, and authorization on every call
- Avoid long synchronous call chains across many services



CONSUMER DESIGN REQUIREMENTS

- Consumers must be idempotent and tolerant of duplicate messages
- Use dead-letter queues for repeated processing failures
- Monitor broker health and message backlog

KEY TAKEAWAY Transaction Service calling Account Service's API is easy to build and test, but both services must be healthy at the same time.

- Account Service publishes AccountOpened without knowing who is listening -- Notification, Reporting, and Fraud services react independently.

HYBRID INTEGRATION MODELS · SERVICE DEPENDENCIES AND DEPENDENCY GRAPHS



COMMON HYBRID PATTERNS

- **Sync request + async event:** handle the request synchronously, then publish an event
- **Async command + sync query:** send a command asynchronously, query status synchronously
- **Event notification + sync lookup:** notify via events, fetch full details via API when needed
- **Async processing + sync callback:** process long-running work async, callback or poll for the result



DESIGN GUIDELINES

- Use synchronous calls for short, user-facing interactions
- Use asynchronous messaging for decoupled, long-running, or fan-out work
- Propagate correlation IDs across both sync and async paths



WHY DEPENDENCY GRAPHS MATTER

- Visualizing call chains reveals services with too many downstream dependencies
- Highlights single points of failure that many services depend on
- Helps teams reason about blast radius before making a change

KEY TAKEAWAY Handle a request synchronously, then publish an event; or send a command asynchronously and query its status synchronously later. • A dependency graph shows which services call which, exposing long chains, single points of failure, and unexpected coupling.

PREVENTING EXCESSIVE SERVICE CHAINING · API COMPOSITION PATTERN · BACKEND-FOR-FRONTEND PATTERN



WHY CHAINING IS RISKY

- A -> B -> C -> D means A's response time depends on every downstream service
- A single slow or failing service in the chain can cascade failure upstream



HOW API COMPOSITION WORKS

- A composition layer calls multiple services (often in parallel) and merges the results
- The client makes one call instead of orchestrating several
- Useful for read-heavy views that combine data from many owning services



WAYS TO REDUCE CHAINING

- Use API composition or a BFF to flatten fan-out calls
- Replace synchronous chains with events where immediacy is not required
- Set aggressive timeouts and circuit breakers on every hop



WHY USE A BFF

- Each client type gets an API shaped for its own needs, not a lowest-common-denominator contract
- Reduces over-fetching and under-fetching for constrained clients like mobile
- Isolates client-specific logic away from core domain services

KEY TAKEAWAY Each synchronous hop in a chain adds latency and a new way for the whole request to fail -- shorten chains or break them with async patterns.

- A composer service calls several downstream services in parallel and merges their results into a single client-facing response.
- Mobile, web, and partner clients often need different shapes of the same data -- a BFF per client type keeps each API simple and fit for purpose.

API GATEWAY PATTERN · SERVICE DISCOVERY CONCEPTS · CENTRALIZED VS. DECENTRALIZED ORCHESTRATION



WHAT AN API GATEWAY PROVIDES

- A single entry point for client traffic into the microservices landscape
- Routing to the correct backend service based on the request
- **Cross-cutting concerns:** authentication, rate limiting, request/response policies



CENTRALIZED COORDINATION

- One coordinator service directs each step of a multi-service workflow
- Easier to visualize, monitor, and reason about the overall flow



WHY SERVICE DISCOVERY MATTERS

- Service instances scale up, scale down, and restart with new network addresses
- A discovery mechanism (registry, DNS, or platform-native discovery) tracks healthy instances
- Callers resolve a logical service name instead of a hard-coded IP address



DECENTRALIZED COORDINATION

- Each service reacts to events and decides its own next step
- More resilient to a single coordinator failing, but harder to trace end to end

KEY TAKEAWAY Centralizing routing, authentication, and policy enforcement at the edge keeps individual services simpler and more consistent.

- In a dynamic, containerized environment, service instances come and go -- discovery keeps callers pointed at healthy, current instances.
- A central orchestrator gives visibility and control; decentralized coordination avoids a single point of control but is harder to trace.

Orchestration vs. Choreography

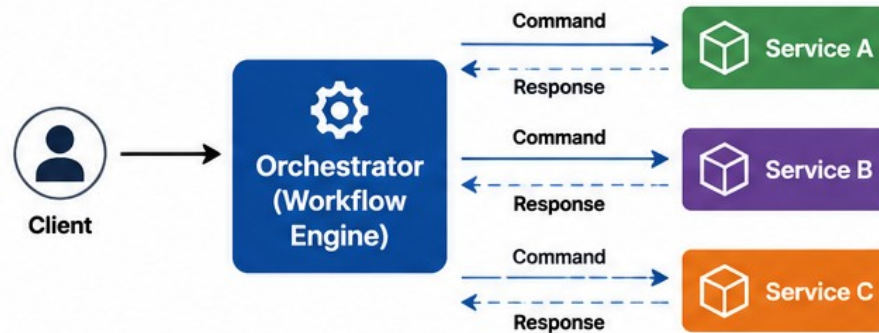
Two styles for coordinating work between microservices.



ORCHESTRATION

Centralized control

A central orchestrator (workflow engine) decides the next step and sends commands to participant services.



How it works: The orchestrator has the process logic, makes decisions, and explicitly calls each service in the required order.



Pros: Central visibility, predictable flow, easier error handling, simpler to monitor and audit.



Cons: Single point of failure, potential bottleneck, tight coupling to the orchestrator, harder to scale.



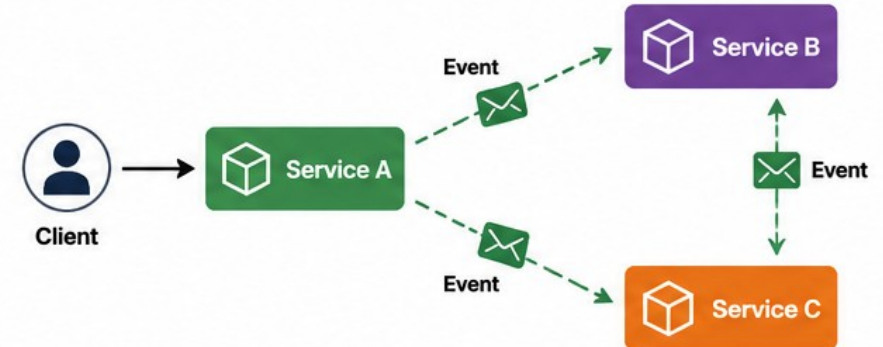
Best for: Complex processes with clear steps, long-running transactions, sagas that need strict control and compensation.



CHOREOGRAPHY

Decentralized coordination

Services collaborate by publishing events and reacting to events from other services.



How it works: Each service publishes events when something happens and listens for events to decide what to do next.



Pros: High scalability, loose coupling, resilient, services can evolve independently, no central bottleneck.



Cons: Harder to visualize the overall flow, eventual consistency, more complex debugging and error handling.



Best for: Event-driven domains, real-time updates, systems that need flexibility and can tolerate eventual consistency.

VS.



Which one should you use?

Choose Orchestration when:

- The process is complex and step-driven
- You need central visibility and control
- You require guaranteed ordering/compensation
- Compliance and auditing are critical



Hybrid Approach

Use orchestration for critical flows and choreography for notifications and updates.

Choose Choreography when:

- The system is event-driven by nature
- You need high scalability and independence
- The flow is dynamic or not fully predictable
- Eventual consistency is acceptable



DESIGNING FOR INDEPENDENT DEPLOYMENT · VERSIONING SERVICES WITHOUT BREAKING CONSUMERS



WHAT INDEPENDENT DEPLOYMENT REQUIRES

- Stable, versioned API and event contracts that consumers can rely on
- No shared database with any other service
- Backward-compatible changes as the default deployment mode
- Automated tests and pipelines scoped to the single service



EVOLVING SERVICE CONTRACTS SAFELY

- Prefer additive, backward-compatible changes whenever possible
- Reserve a new major version for genuinely breaking changes
- Communicate deprecation timelines and support old versions for a defined period

KEY TAKEAWAY Independent deployability is the practical test of good boundaries -- stable contracts, owned data, and backward-compatible changes make it possible. • Backward-compatible changes ship freely; breaking changes get an explicit new version and a communicated deprecation timeline.



Microservices Design Review Checklist



Use this checklist to validate the quality, independence, and operational readiness of your microservices.

1. Business & Domain Alignment

- ☐ Service aligns to a specific business capability.
- ☐ Bounded context is clearly defined.
- ☐ Ubiquitous language is used consistently.
- ☐ Dependencies on other domains are minimized.
- ☐ Service purpose and scope are well documented.

Goal: Solve a real business need within clear boundaries.

2. Service Boundaries & Cohesion

- ☐ Service has high cohesion and single responsibility.
- ☐ Data ownership is clearly defined.
- ☐ No shared database or schema with other services.
- ☐ Granularity is right-sized (not too big / not too small).
- ☐ Domain model and subdomains are respected.

Goal: Independent teams can own and evolve the service.

3. Contracts & Interfaces

- ☐ REST APIs are resource-oriented and consistent.
- ☐ Request/response models are well designed.
- ☐ Validation rules and error responses are defined.
- ☐ API versioning strategy is defined.
- ☐ OpenAPI/Swagger documentation is available.

Goal: Clear, stable contracts for all consumers.

4. Data & Persistence

- ☐ Service owns its data and schema.
- ☐ Data access is encapsulated by the service.
- ☐ Migrations are managed (Flyway/Liquibase).
- ☐ Transactions are local to the service.
- ☐ Caching strategy is appropriate (if used).

Goal: Data autonomy and consistency.

Design Readiness Scorecard

Business Alignment ___/5

Boundaries & Cohesion ___/5

Contracts & Interfaces ___/5

Data & Persistence ___/5

Communication ___/5

Resilience ___/5

Security ___/5

Observability ___/5

Operations & Deployment ___/5

Documentation & Governance ___/5

Total Score: ____ / 50

45–50 = Excellent | 35–44 = Good

25–34 = Needs Improvement | <25 = At Risk

5. Communication & Integration

- ☐ Communication style (sync/async) is chosen intentionally.
- ☐ APIs/events are versioned and backward-compatible.
- ☐ Message schemas are defined and versioned.
- ☐ Handling of partial failures is defined.
- ☐ External dependencies are minimal and documented.

Goal: Reliable, decoupled integration.

6. Resilience & Fault Tolerance

- ☐ Timeouts are configured appropriately.
- ☐ Retries with backoff are used where safe.
- ☐ Circuit breakers are applied for dependencies.
- ☐ Bulkheads / rate limits are in place.
- ☐ Fallback and degradation strategies exist.

Goal: Service continues to operate under failure.

7. Security

- ☐ Authentication and authorization are defined.
- ☐ Service-to-service authentication is secure.
- ☐ Sensitive data is protected (at rest & in transit).
- ☐ Secrets are managed securely.
- ☐ Input validation and output sanitization are in place.

Goal: Secure by design and by default.

8. Observability

- ☐ Structured logging is implemented.
- ☐ Metrics are exposed (business & technical).
- ☐ Distributed tracing is enabled.
- ☐ Health checks (liveness/readiness) are defined.
- ☐ Alerts and dashboards are planned.

Goal: Operate, monitor and troubleshoot effectively.

9. Operations & Deployment

- ☐ Service is containerized and immutable.
- ☐ Configuration is externalized.
- ☐ Can be deployed independently.
- ☐ Supports horizontal scaling.
- ☐ CI/CD pipeline exists for the service.

Goal: Independent delivery and operations.

10. Documentation & Governance

- ☐ Architecture and design decisions are documented.
- ☐ Runbooks / operational guides exist.
- ☐ Service ownership and team model are defined.
- ☐ Dependencies and change impact are understood.
- ☐ Governance and review process is followed.

Goal: Maintainable and governed at scale.



Key Questions to Ask

- Can this service be developed, tested and deployed independently?
- What happens if a dependent service is slow or down?
- Will a change in this service break any existing consumers?
- Do we own our data? Can we evolve our schema safely?
- Is this service observable, secure and resilient by design?
- Is the team equipped to operate this service in production?



Red Flags

- Shared database or tables
- Tight coupling / chatty communication
- Unversioned APIs or events
- Lack of monitoring and alerts
- No clear owner or team
- Manual or non-reproducible deployments



Design for Change. Build for Independence.
Operate with Confidence.



Small, Autonomous Teams

Enable ownership
end-to-end.



Evolve Safely

Use versioning and
backward-compatible changes.



Automate Everything

CI/CD, tests, and
infrastructure as code.



Measure & Improve

Learn from metrics,
incidents, and feedback.

REST ARCHITECTURAL PRINCIPLES · RESOURCES, REPRESENTATIONS, AND IDENTIFIERS



REST PRINCIPLES

- **Resource-oriented:** clients interact with nouns, not actions
- **Stateless:** each request carries enough information to be processed on its own
- Consistent, predictable, HTTP-native, and cache-aware where appropriate



THREE CORE REST CONCEPTS

- **Resources:** nouns such as Users, Orders, Products, Payments, Shipments
- **Representations:** JSON, XML, HTML, CSV -- the client chooses via content negotiation
- **Identifiers:** stable, persistent URIs such as /orders/123 or /customers/7/orders



GOOD VS. POOR REST DESIGN

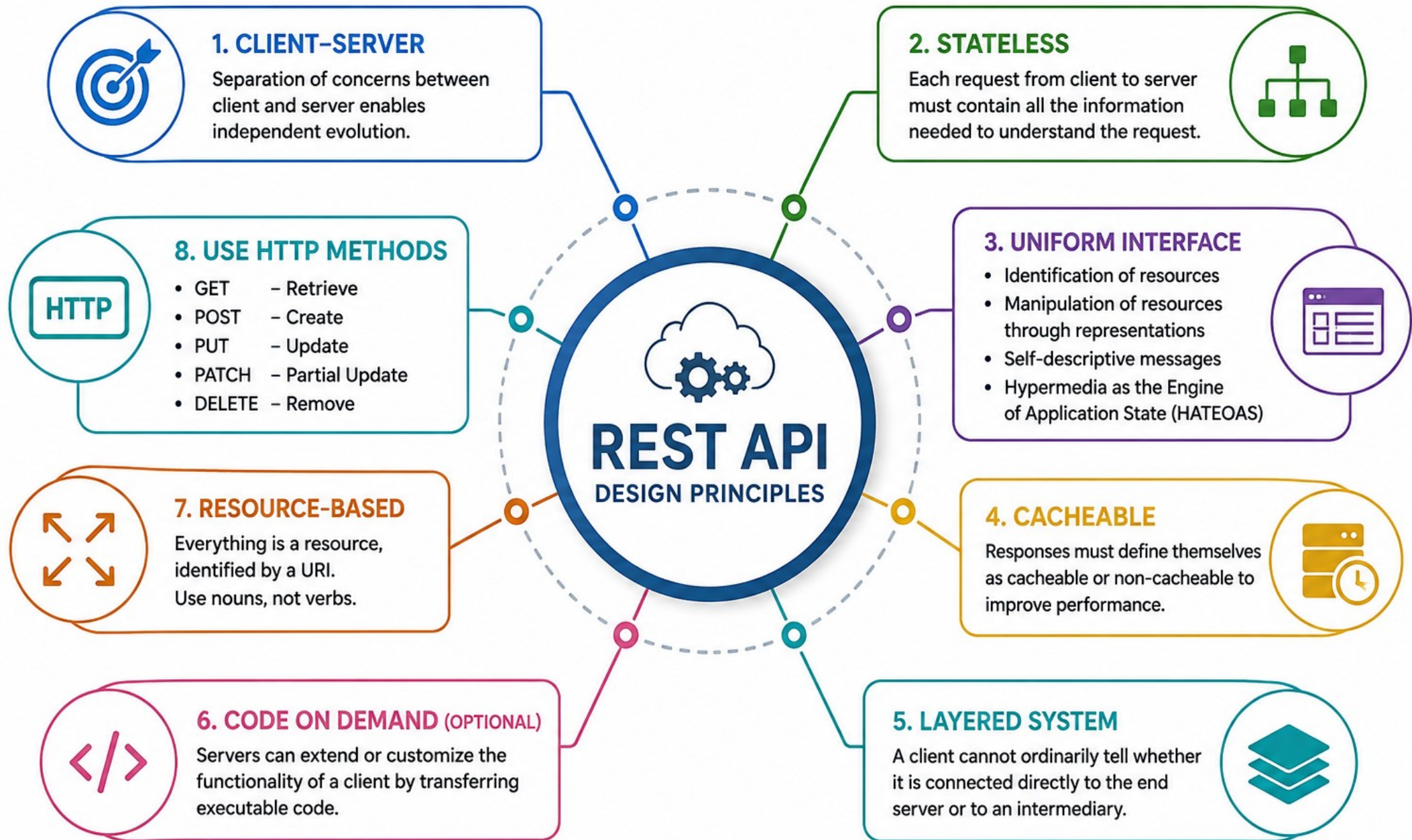
- **Good:** GET /customers/101, POST /customers, GET /accounts/5001/transactions
- **Poor:** /getCustomer, /createAccount, /fetchTransactions, /deleteLoan
- The URI identifies the resource; the HTTP method expresses the action



GOOD VS. BAD IDENTIFIERS

- **Good:** /orders/{id}, /customers/{id}/orders, /products/{sku}
- **Avoid:** /getOrderByid/123, /order?id=123, /createUser, /updateProductInfo

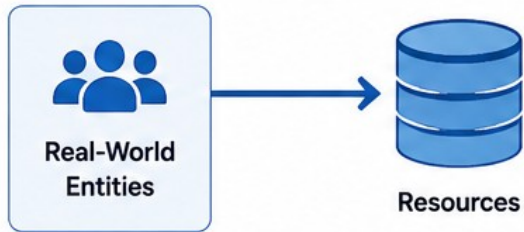
KEY TAKEAWAY REST is popular for microservices because it is simple, language-independent, HTTP-native, and easy to test. • A resource is the thing itself, a representation is how it's transferred (JSON, XML), and an identifier (URI) is how it's found.



RESOURCE MODELING AND URI DESIGN

1. RESOURCE MODELING

Model real-world entities as resources.



2. RESOURCE CHARACTERISTICS



Identifiable

Each resource is uniquely identifiable.



Stateful

Resources have state represented in server.



Addressable

Resources are accessed via URIs.



Representable

Resources are returned in various representations (JSON, XML, etc.).

3. URI DESIGN PRINCIPLES



Use nouns, not verbs

Use resource names, not actions.



Hierarchical structure

Show relationships using nesting.



Use lowercase letters

URIs are case-sensitive.



Use hyphens to separate words

Improve readability.



Use numbers for IDs

Use simple, stable identifiers.



Keep URIs simple and consistent

Avoid unnecessary complexity.

4. URI STRUCTURE

Scheme

Domain

Resource Path

Query

https://

api.example.com

/users/123/orders

?active=true&page=1

Protocol
(HTTPS)

Server
(API Host)

Resource Path
(Identifies
Resource)

Query Parameters
(Optional
Filters, Sorting)

5. EXAMPLES

Description

URI Example



Collection of users

GET /users



Specific user

GET /users/123



Orders of a user

GET /users/123/orders



Specific order

GET /users/123/orders/456



Comments of an order

GET /users/123/orders/456/comments

6. RELATIONSHIP REPRESENTATION

One-to-One



/users/123/profile

One-to-Many



/users/123/orders

Many-to-Many



/users/123/roles

Nested Resources



/users/123/orders/456/items

Principles of Resource-Oriented URLs



1. Use Nouns, Not Verbs

Resources are nouns (e.g., users, orders). HTTP methods define the action.



2. Represent Resources as Collections

Use plural nouns for collections. Use singular nouns for individual resources.



3. Hierarchy Through Relationships

Use path segments to show relationships. Reflect real-world or domain relationships.



4. Use Identifiers in the Path

Identify a specific resource using its unique ID.



5. Keep It Simple and Consistent

Use lowercase, hyphens for readability. Be consistent across your API.



6. Avoid Implementation Details

Do not expose verbs, actions, or internal logic. Avoid file extensions.

Examples: Good vs. Bad URLs



Good (Resource-Oriented)



/users



/users/42



/users



/users/42



/users/42



/users/42/orders



/orders/123/items



/orders/123



Bad (Action-Oriented)



/getAllUsers



/getUserById/42



/createUser



/updateUser/42



/deleteUser/42



/getOrdersForUser/42



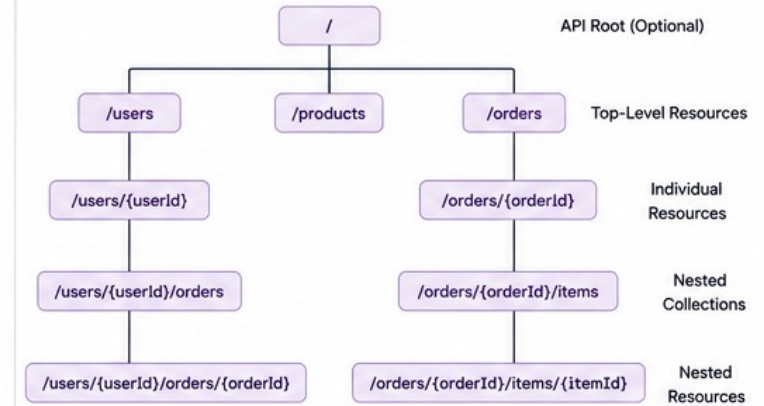
/getOrderItems/123



/updateOrderStatus/123

Building Hierarchical URLs

Example Domain: E-Commerce API



Path Design Best Practices



Use plural nouns for collections
/users not /user



Use hyphens (-) to separate words
/user-profiles not /user_profiles



Use singular nouns for individual resources
/users/42 not /users/42/details



Use lowercase letters
/orders not /Orders



Use meaningful, domain-specific names
/products not /items



Do not include file extensions
/users not /users.json

Applying HTTP Methods to URLs

Method	Purpose	Example URL
GET	Retrieve a resource or collection	/products /products/101
POST	Create a new resource	/products
PUT	Replace an entire resource	/products/101
PATCH	Partially update a resource	/products/101
DELETE	Delete a resource	/products/101

Complex Query Use Cases



Filtering
/products?category=books



Sorting
/products?sort=price,asc



Pagination
/products?page=2&size=20



Field Selection
/products?fields=id,name,price



Search
/products?search=rest+api



Keep query parameters for:

- Filtering
- Sorting
- Pagination
- Field selection
- Searching



Do not use query parameters for resource relationships or identifiers.



Checklist



Resources are nouns (plural for collections)



Hierarchical relationships are clear



Identifiers are in the path (not in query)



Consistent naming (lowercase, hyphens)



No verbs or actions in the URL



No file extensions



Stable and predictable URL structure



Easy to read and intuitive



Query parameters used only for filtering, sorting, pagination, search, etc.



HTTP Methods and Their Semantics

HTTP methods define the type of operation to be performed on a resource. Choosing the right method leads to clear, predictable, and RESTful APIs.



Method	Purpose	Semantic Details	Idempotent	Safe	Typical Status Codes
GET	Retrieve a resource	Requests data from the server. Should not change server state.	✓ Yes	✓ Yes	200 OK, 204 No Content, 304 Not Modified, 404 Not Found
POST	Create a new resource	Submits data to the server to create a new resource. Not idempotent by default.	✗ No	✗ No	201 Created, 202 Accepted, 400 Bad Request, 409 Conflict
PUT	Replace a resource	Replaces the entire resource at the target URI with the provided data. Idempotent.	✓ Yes	✗ No	200 OK, 204 No Content, 400 Bad Request, 404 Not Found
PATCH	Partially update a resource	Applies partial modifications to a resource. Not idempotent by default.	✗ No	✗ No	200 OK, 204 No Content, 400 Bad Request, 404 Not Found
DELETE	Delete a resource	Removes the resource at the target URI. Idempotent (deleting multiple times has the same effect).	✓ Yes	✗ No	200 OK, 204 No Content, 404 Not Found
HEAD	Retrieve headers only	Same as GET, but returns only headers. No response body.	✓ Yes	✓ Yes	200 OK, 304 Not Modified, 404 Not Found
OPTIONS	Discover supported options	Returns the HTTP methods supported by the server for the target resource.	✓ Yes	✓ Yes	200 OK



Key Takeaways

- Use the most specific method that matches your intent.
- Safe methods (GET, HEAD, OPTIONS) must not change server state.
- Idempotent methods (GET, PUT, DELETE, HEAD, OPTIONS) can be repeated without changing the result beyond the first time.
- Clients, proxies, and caches rely on correct method semantics.



Example Resource: /api/orders

GET	/api/orders/123	– Get order 123
POST	/api/orders	– Create a new order
PUT	/api/orders/123	– Replace order 123
PATCH	/api/orders/123	– Update part of order 123
DELETE	/api/orders/123	– Delete order 123
HEAD	/api/orders/123	– Get headers for order 123
OPTIONS	/api/orders	– Discover allowed methods



SAFE AND IDEMPOTENT OPERATIONS

All safe methods are idempotent, but not all idempotent methods are safe -- PUT changes state safely on repeat, GET never changes it at all.



SAFE OPERATIONS

- Does not change server state -- intended for read-only operations
- Multiple identical requests have the same effect as a single request
- Safe methods can be cached, prefetched, and retried automatically
- **Safe methods:** GET, HEAD, OPTIONS



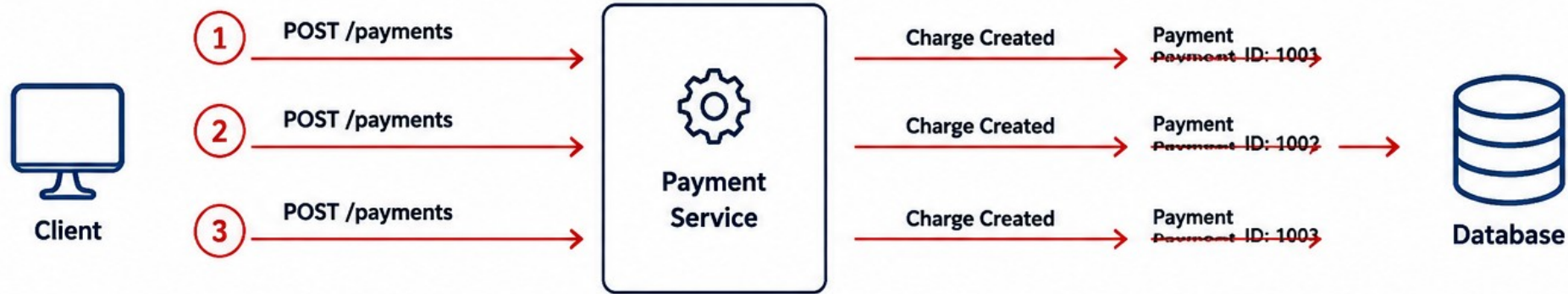
IDEMPOTENT OPERATIONS

- Making the same request multiple times has the same effect as making it once
- Enables safe retries when a response is lost or the client is unsure
- **Idempotent methods:** GET, HEAD, PUT, DELETE, OPTIONS
-- POST is neither safe nor idempotent by default

KEY TAKEAWAY All safe methods are idempotent, but not all idempotent methods are safe -- PUT changes state safely on repeat, GET never changes it at all.

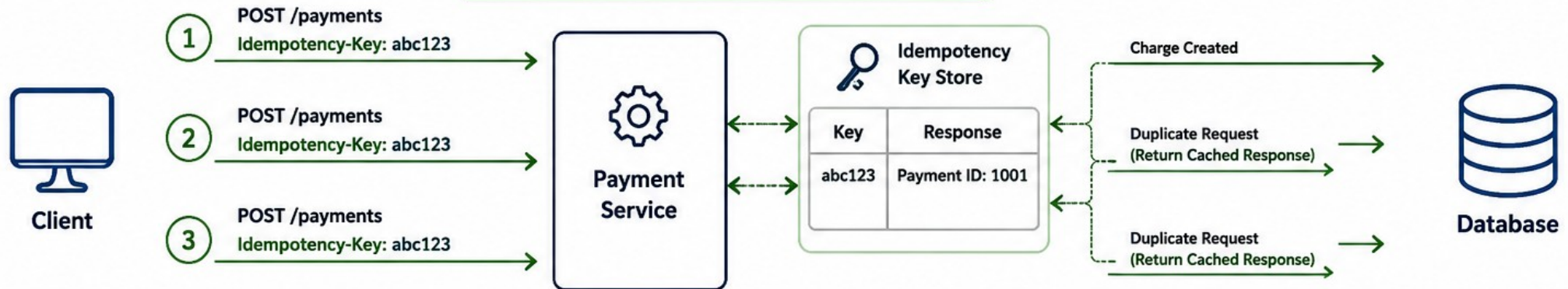
Idempotency

Non-Idempotent (Without Idempotency Key)



Result: Multiple charges created for the same request ✗

Idempotent (With Idempotency Key)



Result: Only one charge created, same response returned for retries ✓

HTTP STATUS CODES FOR MICROSERVICES · REQUEST AND RESPONSE DESIGN

STATUS CODE CATEGORIES

RANGE	CATEGORY	EXAMPLES
2xx	Success	200 OK, 201 Created, 204 No Content
4xx	Client Error	400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable



BEST PRACTICES

- Use the most accurate status code -- never return 200 OK for every response
- Provide meaningful error details and a consistent error body across services
- Never expose stack traces or internal details in an error response
- Include a correlation ID in every error response for traceability



REQUEST KEY ELEMENTS

- **Method & URL:** the correct HTTP method and resource-oriented URL
- **Headers:** authentication, content type, correlation ID, idempotency key
- **Body:** structured, validated JSON with clear fields



REQUEST & RESPONSE BEST PRACTICES

- **Be consistent:** same naming, casing, and structure across endpoints
- **Be explicit:** descriptive field names, only required fields
- **Make it traceable:** propagate correlation IDs, use idempotency keys for safe retries

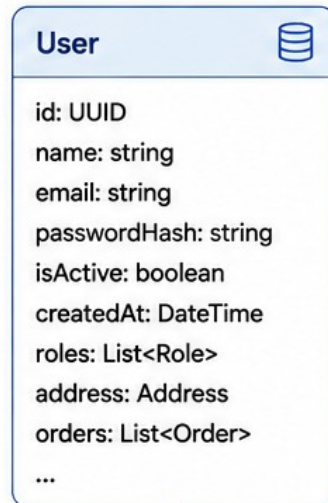
KEY TAKEAWAY Choosing the accurate status code, not always 200 OK, is what lets clients handle success, client errors, and server errors correctly. • Every request and response should be efficient, use the correct HTTP method and status code, and carry a correlation ID for tracing.

DTOs and API Contracts

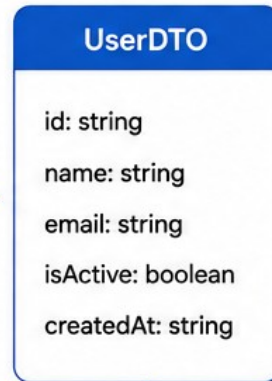
1. DTOs (Data Transfer Objects)

DTOs are simple objects used to transfer data between layers or services.

Internal Domain Model
(Complex)



DTO



- Hides internal structure
- Exposes only required data
- Prevents over-posting
- Improves security & versioning

Lightweight object used for data transfer across boundaries

2. API Contracts

An API Contract defines the structure, format, and rules of request and response between client and server.



Client

HTTP Request
(JSON)

POST /api/users

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "isActive": true
}
```



Server

HTTP Response
(JSON)

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "John Doe",
  "email": "john@example.com",
  "isActive": true,
  "createdAt": "2024-05-08T12:30:00Z"
}
```

Internal
Domain Model

Contract Includes



Endpoints
(URI)



HTTP Methods



Request Schema
(DTOs)



Response Schema
(DTOs)



Validation Rules



Error Codes



Benefits



Loose Coupling

Client and server can evolve independently



Consistency

Well-defined contracts ensure predictable interactions



Maintainability

Changes are managed through versioned contracts



Security

Exposes only necessary data and validation rules

JSON PAYLOAD DESIGN GUIDELINES · API VALIDATION AND ERROR HANDLING



GUIDING PRINCIPLES

- Be consistent in naming, structure, and data types across all APIs
- Use clear, descriptive names that reflect the domain, not abbreviations
- **Keep it simple:** include only necessary fields, avoid deep nesting
- **Design for evolution:** add new fields without breaking existing consumers



VALIDATION FLOW

1. Request received from the client
2. Validate syntax, structure, and business rules
3. Is it valid?
4. **If yes:** process the request and return a success response
5. **If no:** return a standardized error response with details



BEST PRACTICES

- Use camelCase property names and plural names for collections
- Use ISO 8601 format for dates, and IDs as strings even when numeric
- Use null sparingly -- prefer omitting a field over sending null



ERROR HANDLING PRINCIPLES

- Use a consistent error format across all services
- Provide clear, actionable messages that help the client fix the issue
- Never expose stack traces or sensitive internals
- Log errors with correlationId and sufficient context

KEY TAKEAWAY Clear naming, the right data types, ISO 8601 dates, and no unnecessary nesting turn a JSON payload into a payload teams can trust. • A five-step validation flow -- receive, validate, check, process or reject -- keeps bad data out and gives clients actionable errors.

STANDARDIZED API ERROR RESPONSES · PAGINATION, FILTERING, AND SORTING



STANDARD ERROR RESPONSE FIELDS

- timestamp, status, error, message -- when, what code, and why
- path -- the request path that caused the error
- errorCode -- an application-specific code for programmatic handling
- correlationId -- ties the error back to logs and traces
- details / fieldErrors -- field-level validation messages when relevant



THREE QUERY CAPABILITIES

- **Pagination:** page/size or cursor-based, e.g. ?
page=2&size=20&sort=orderDate,desc
- **Filtering:** operators like eq, ne, gt, lt, in, like, between, e.g. ?
status=in:NEW,PAID
- **Sorting:** sort=field,direction, supporting multiple fields for stable sort order



BEST PRACTICES

- Always paginate large datasets -- never return unbounded lists
- Set a maximum page size to prevent abuse (for example, max 100)
- Return pagination metadata (totalElements, hasNext) in every paginated response
- Whitelist allowed fields for filtering and sorting

KEY TAKEAWAY Every error response should carry a timestamp, status, message, path, error code, and correlation ID so clients can handle it reliably. • Query parameters for page, filter operators, and sort direction keep large collections usable, performant, and predictable.

PARTIAL UPDATES WITH HTTP PATCH · CONTENT NEGOTIATION



WHY USE PATCH

- **Partial update:** send only the fields that need to change
- Smaller payloads, less bandwidth, and lower risk of overwriting unrelated fields
- Works well with optimistic locking via ETags for safe concurrent updates



HOW CONTENT NEGOTIATION WORKS

1. Client sends a request with an Accept header listing supported media types
2. Server evaluates and selects the best match it can produce
3. Server responds with the selected representation and a matching Content-Type
4. Client consumes the response in the agreed format



COMMON PATCH FORMATS

- **JSON Patch (RFC 6902):** operation-based -- {"op": "replace", "path": "/price", "value": 1099.99}
- **JSON Merge Patch (RFC 7386):** document-based partial body; fields set to null are removed
- PATCH is not idempotent by default -- design idempotent operations when possible



BEST PRACTICES

- Always specify and honor Content-Type in responses
- Prefer JSON as the default representation unless clients need something else
- Use the Vary header for correct caching and proxy behavior

KEY TAKEAWAY JSON Patch and JSON Merge Patch let a client update one or two fields on a resource without resending the entire representation. • The Accept header tells the server what formats a client can handle; the server picks the best match and returns it.

API VERSIONING STRATEGIES

URI, query parameter, header, media type, and domain versioning each solve the same problem with different tradeoffs.



FIVE COMMON VERSIONING STRATEGIES

- **URI path versioning:** /v1/orders/123 -- clear and explicit, widely understood
- **Query parameter versioning:** /orders/123?version=2 -- keeps the base URL clean
- **Header versioning:** Accept-Version: 2 -- keeps the URL itself clean
- **Media type versioning:** Accept: application/vnd.myapi.order+json;v=2
- **Domain versioning:** v1.api.example.com -- strong separation, more operational overhead



WHEN TO VERSION

- **Version for breaking changes:** removed fields, changed types, new required fields
- Do not version for additive, backward-compatible changes

KEY TAKEAWAY URI, query parameter, header, media type, and domain versioning each solve the same problem with different tradeoffs.

API Versioning Overview



What is API Versioning?

API versioning is a strategy to manage changes to an API over time in a backward-compatible way, enabling multiple API versions to coexist.

Why Version APIs?



Backward Compatibility

Existing clients continue to work without breaking changes.



Client Flexibility

Clients can choose when to upgrade to a newer version.



Evolve Safely

Introduce new features and improvements safely.



Better Lifecycle Management

Easier to deprecate and sunset older versions.

Common API Versioning Strategies

1. URI Path Versioning



Example

```
GET /api/v1/customers
GET /api/v2/customers
```

- ✓ Clear and explicit
- ✗ Multiple URLs to maintain

2. Query Parameter Versioning



Example

```
GET /api/customers?version=1
GET /api/customers?version=2
```

- ✓ Simple to implement
- ✗ Easy to forget version param

3. Custom Header Versioning



Example

```
Header: X-API-Version: 1
Header: X-API-Version: 2
```

- ✓ Keeps URI clean
- ✗ Less visible to developers

4. Media Type Versioning



Example

```
Accept: application/vnd.api+json;v=1
Accept: application/vnd.api+json;v=2
```

- ✓ RESTful and flexible
- ✗ More complex to implement

Versioning Lifecycle



1. Release

Release a new API version.



2. Communicate

Notify developers about changes and new version.



3. Deprecate

Mark old version as deprecated with sunset timeline.



4. Sunset

Retire old version after migration period.

Best Practices



Version when you make breaking changes.



Document changes clearly for each version.



Provide a deprecation policy and sunset timeline.



Automate tests across multiple versions.



Monitor usage of different versions.



Support and guide clients through migration.

BACKWARD-COMPATIBLE API EVOLUTION · IDEMPOTENCY KEYS



SAFE (BACKWARD-COMPATIBLE) CHANGES

- Add new optional fields to requests or responses
- Add new resources or endpoints that don't affect existing ones
- Add new enum values that old clients can gracefully ignore



CHANGES THAT BREAK COMPATIBILITY

- Removing or renaming fields or endpoints
- Changing the meaning, type, or default behavior of a field
- Making a previously optional field required



HOW IDEMPOTENCY KEYS WORK

1. Client sends a unique Idempotency-Key header with the request
 2. Server checks whether that key has already been processed
 3. **If new:** process the request and store the response keyed by that value
 4. **If it exists:** return the original stored response instead of reprocessing
- Payment processing -- avoid double charges on retry
 - Order creation -- prevent duplicate orders from network retries
 - Scope keys to the resource or tenant and set a reasonable TTL value



COMMON USE CASES

KEY TAKEAWAY New optional fields and new endpoints are safe; removing fields, renaming fields, or changing meaning are breaking changes that need a version. • Same key, same result: a client-generated idempotency key lets the server return the original response instead of processing a duplicate.

CORRELATION AND TRACE IDENTIFIERS · API DOCUMENTATION WITH OPENAPI

WHAT AND WHY

- Correlation and trace identifiers uniquely tag a request as it flows across services
- They let teams correlate logs, metrics, and traces for faster troubleshooting
- Essential when a single user request touches many services

WHAT OPENAPI PROVIDES

- A standard (formerly Swagger) format for describing REST APIs
- Documents endpoints, methods, request/response schemas, and status codes
- Powers interactive documentation (Swagger UI), client generation, and contract testing

BEST PRACTICES

- Generate the trace ID at the edge (API gateway or entry point)
- Propagate it in HTTP headers and messaging metadata everywhere downstream
- Log the trace ID consistently in every log statement across every service

BEST PRACTICES

- Keep the spec versioned alongside the code
- Define all responses, including errors, with clear descriptions
- Validate the spec in CI/CD (for example, with Spectral)

KEY TAKEAWAY A trace ID created at the API gateway and propagated through every downstream call turns scattered logs into one connected story. • Define, serve, visualize, automate, and collaborate: OpenAPI documentation keeps a REST API discoverable and its contract enforceable.

OpenAPI Documentation



OpenAPI (formerly Swagger) is a standard, language-agnostic specification for describing RESTful APIs. It enables documentation, tooling, automation, and client generation.

Key Benefits



Standardized

A vendor-neutral standard for API description.



Machine-Readable

Enables automation, code generation, and validation.



Consistent

Defines a consistent way to document APIs.



Better Collaboration

Improves communication between teams and stakeholders.



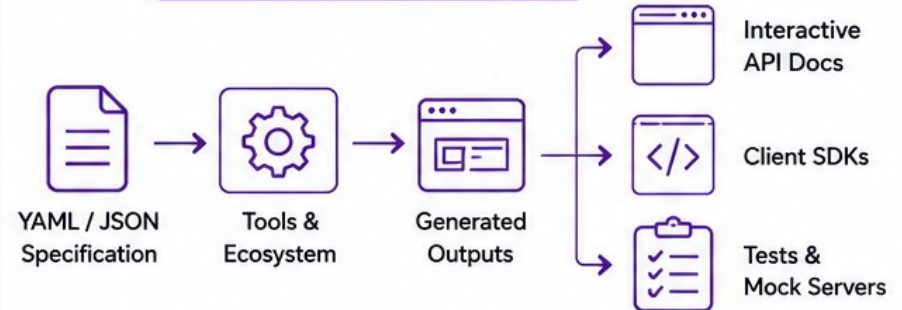
Tooling Ecosystem

Works with a rich ecosystem of tools (UI, testing, mocking, code generation).

OpenAPI Specification (Example)

```
openapi: 3.0.3
info:
  title: Banking API
  version: 1.0.0
  description: API for managing bank accounts
servers:
  - url: https://api.bank.com/v1
    description: Production server
paths:
  /accounts:
    get:
      summary: Get all accounts
      description: Returns a list of all accounts
      responses:
        '200':
          description: Successful response
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Account'
components:
  schemas:
    Account:
      type: object
      properties:
        id:
          type: string
        accountNumber:
          type: string
        balance:
          type: number
          format: double
        currency:
          type: string
      required: [id, accountNumber, balance, currency]
```

How It Works



Common Tools



Swagger UI
(OpenAPI Editor)



Swagger Codegen
/ OpenAPI Generator



Postman
(Import OpenAPI)



Stoplight
Studio

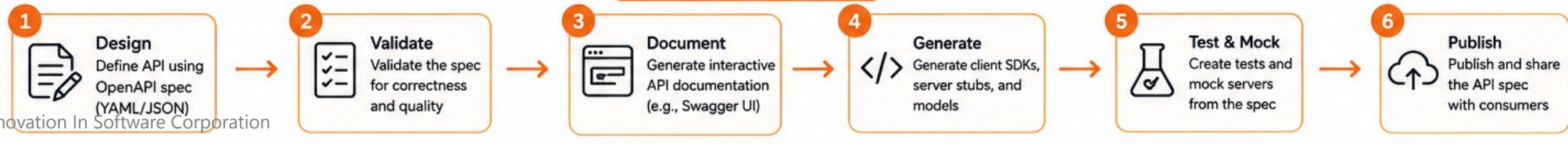


Redoc
(Documentation)



Prism
(Mock Server)

Typical Workflow



CONTRACT-FIRST VS. CODE-FIRST API DEVELOPMENT · CONSUMER-DRIVEN API DESIGN



CONTRACT-FIRST

- Define the OpenAPI contract, review and agree with consumers, then implement
- Strong collaboration and alignment; enables true parallel development
- Best when multiple teams or external consumers depend on the API



CODE-FIRST

- Implement the API in code, then generate the OpenAPI spec from it
- Faster start, documentation always reflects the code
- Best for internal, simple CRUD-style APIs owned end to end by one team



THE CDAD CYCLE

- 1. Consumer identifies a need:** a use case and required data
- 2.** Consumer writes a contract defining the expected request/response
- 3.** Provider implements the API to satisfy the contract
- 4.** Provider runs contract verification tests to confirm compliance
- 5.** Deploy and monitor, feeding real usage back into future contracts

KEY TAKEAWAY Contract-first suits multi-team, business-critical APIs; code-first suits fast-moving internal APIs owned end to end by one team.

- Consumers define the contract as executable tests; providers implement to satisfy it, catching breaking changes before deployment.

REST API SECURITY CONSIDERATIONS · REST API DESIGN ANTI-PATTERNS



COMMON API THREATS

- Broken authentication and broken authorization (IDOR)
- Injection attacks, sensitive data exposure, security misconfiguration
- Insufficient logging and monitoring



ANTI-PATTERNS TO AVOID (1 OF 2)

- CRUD-everywhere naming (/getUsers) instead of resource nouns
- Over-using query parameters instead of clean filters and pagination
- Exposing internal database structures directly in responses
- Ignoring HTTP status codes and always returning 200 OK
- Over- or under-fetching data with no field selection
- No pagination on large collections



SECURE API DESIGN PRINCIPLES

- Validate everything -- all inputs, headers, and parameters
- Use HTTPS everywhere and enforce TLS 1.2 or higher
- Minimize data exposure -- return only what's necessary
- Rate limit and throttle to prevent abuse



ANTI-PATTERNS TO AVOID (2 OF 2)

- Inconsistent naming, casing, and error formats across endpoints
- Using GET for operations that have side effects
- Tunneling every verb through POST with a _method override
- Leaking sensitive information in stack traces or error details
- Versioning in the wrong way, locking the API into one rigid scheme
- Chatty, overly fine-grained APIs that require many round trips

KEY TAKEAWAY Authenticate, authorize, validate, and encrypt at every layer -- from the client request through business logic to data access. • From CRUD-everywhere naming to always returning 200 OK, each anti-pattern has a simple, resource-oriented fix.

REST API Design Checklist

Use this checklist to design REST APIs that are consistent, secure, scalable, and easy to use.



1. RESOURCE DESIGN

- ☐ Use nouns, not verbs, in URLs
- ☐ Use plural nouns for collections
- ☐ Use hierarchical structure to show relationships
- ☐ Keep URLs short, clear, and readable
- ☐ Use consistent naming conventions
- ☐ Version your API (if needed)
- ☐ Avoid exposing internal details in URLs

2. HTTP METHODS

- ☐ Use correct HTTP methods for operations
- ☐ GET: Retrieve data (safe, idempotent)
- ☐ POST: Create new resource
- ☐ PUT: Replace entire resource (idempotent)
- ☐ PATCH: Partial update
- ☐ DELETE: Remove resource (idempotent)
- ☐ Avoid using GET for operations with side effects

3. REQUEST & RESPONSE

- ☐ Use JSON as the default format
- ☐ Accept and return the correct Content-Type
- ☐ Keep request and response structures consistent
- ☐ Use clear and meaningful property names
- ☐ Return only necessary data
- ☐ Include pagination, filtering, and sorting
- ☐ Use proper status codes

4. STATUS CODES

- ☐ 2xx for success (200, 201, 204)
- ☐ 4xx for client errors (400, 401, 403, 404, 409, 422)
- ☐ 5xx for server errors (500, 502, 503, 504)
- ☐ Return appropriate code for all scenarios
- ☐ Do not return 200 OK for errors
- ☐ Include meaningful error details in the response

5. ERROR HANDLING

- ☐ Return consistent error format
- ☐ Provide machine-readable error codes
- ☐ Include human-readable error messages
- ☐ Include correlation/trace ID in error responses
- ☐ Do not expose stack traces or sensitive details

6. QUERY PARAMETERS

- ☐ Use query params for filtering, searching, and sorting
- ☐ Use meaningful and consistent parameter names
- ☐ Provide default values where appropriate
- ☐ Validate all input parameters
- ☐ Document all supported query parameters

7. DATA VALIDATION

- ☐ Validate all input data (server-side)
- ☐ Use proper data types, length, and format checks
- ☐ Return 400 Bad Request for invalid input
- ☐ Provide clear validation error messages
- ☐ Use standardized validation rules

8. SECURITY

- ☐ Use HTTPS for all API calls
- ☐ Authenticate all requests
- ☐ Authorize access to resources
- ☐ Follow least privilege principle
- ☐ Validate and sanitize all inputs
- ☐ Prevent common attacks (SQLi, XSS, CSRF)
- ☐ Do not expose sensitive data in responses

9. DOCUMENTATION

- ☐ Document all endpoints, methods, params, and responses
- ☐ Use OpenAPI (Swagger) specification
- ☐ Provide examples for requests and responses
- ☐ Document error codes and messages
- ☐ Keep documentation up to date
- ☐ Make API docs easy to discover and consume

10. PERFORMANCE

- ☐ Support pagination for large result sets
- ☐ Allow field selection if needed
- ☐ Use caching where appropriate
- ☐ Optimize queries and database access
- ☐ Return compressed responses (GZIP)
- ☐ Monitor and test performance

11. USABILITY & CONSISTENCY

- ☐ Follow consistent naming, casing, and formatting
- ☐ Use predictable and intuitive URLs
- ☐ Keep request/response structures consistent
- ☐ Maintain backward compatibility
- ☐ Deprecate gracefully and communicate changes

12. OBSERVABILITY

- ☐ Include correlation/trace ID in all responses
- ☐ Log requests and responses appropriately
- ☐ Expose metrics and health endpoints
- ☐ Monitor errors, latency, and throughput
- ☐ Set up alerts for failures and anomalies



KEY TAKEAWAY A well-designed API is easy to use, secure, scalable, and a pleasure for developers to integrate.



REMEMBER Design for your consumers. Follow standards. Test thoroughly. Keep improving.

EXERCISE 1.2 — DESIGN ACCOUNT SERVICE APIS AND BOUNDARIES

Module 2 checkpoint B — REST resources, status transitions, OpenAPI sketch.



OBJECTIVE

- REST resources, status transitions, OpenAPI sketch
- 15 min · design checkpoint
- **Worksheet:**
curriculum/day-01/exercises/exercise-1.2-account-apis.md



DO THIS

1. Resource is an account. Sketch POST / GET / PATCH and activate / freeze / close (no DELETE)
2. **Status machine: PENDING → ACTIVE → FROZEN or CLOSED; freeze PENDING is 409; FROZEN can activate; retries return 200**
3. List 400 / 404 / 409 cases; note OpenAPI + Actuator as Lab 1 deliverables
4. **Draw the bounded context:** Account owns accounts only — not transactions, not risk scores



EXPECTED RESULT

- A one-page Account API sketch that Lab 1 can implement without changing URLs tomorrow.
- Use this contract in Lab 1. Do not add Customer or Notification services.

KEY TAKEAWAY A one-page Account API sketch that Lab 1 can implement without changing URLs tomorrow.